DATE: 10/10/2026

CO1-AT1-APPLICATION BASED QUESTION

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG NO: 192324235 **NAME:** JOTHIKA M

ANSWERS:

Section 1: Regular Expressions - Resume Information Extraction

Algorithm

1. Import the Regular Expression module using import re.
2. Store the resume details as text.
3. Extract the candidate's name using r"Name:\s*(.*)".
4. Extract the email address using r"[\w.-]+@[\w.-]+\.\w+".
5. Extract the mobile number using r"\b[6-9]\d{9}\b".
6. Detect technical skills using r"Python|Java|SQL|Machine Learning|NLP".
7. Extract years of experience using r"Experience:\s*(\d+)".
8. Display the extracted candidate details.
9. Check whether the candidate has at least 2 years of experience and Python skill.
10. Display the eligibility status.
11. Stop the program.

CODE:

```
import re
resume = """Name:jothika
Email: jothika@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL
Experience: 3 years"""
name = re.search(r"Name:\s*(.*)", resume).group(1)
email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume).group()
mobile = re.search(r"\b[6-9]\d{9}\b", resume).group()
skills = re.findall( r"Python|Java|SQL|Machine Learning|NLP", resume)
exp = int(re.search(r"Experience:\s*(\d+)", resume).group(1))
```

```
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", ", ".join(skills))
print("Experience:", exp, "years")
if exp >= 2 and "Python" in skills:
    print("Eligibility: Eligible")
else:
    print("Eligibility: Not Eligible")
```

OUTPUT:

Name: jothika
Email: jothika@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL
Experience: 3 years
Eligibility: Eligible

Result Analysis

- Resume Input: The program stores the candidate's name, email, mobile number, skills, and experience as text.
- Name Extraction: The regular expression identifies the Name: field and extracts the candidate's name.
- Email Identification: The program detects the email address using the standard email pattern.
- Mobile Extraction: The program identifies a valid ten-digit Indian mobile number.
- Skill Detection: It finds the specified technical skills, such as Python, Java, and SQL.
- Experience Extraction: The program extracts the candidate's years of experience and converts the value into an integer.
- Eligibility Checking: The candidate is eligible if they have at least two years of experience and Python skill.
- Result: jothika is identified as eligible because he has three years of experience and Python skill.

Section 2: Regular Expressions - Product Search System

Algorithm

1. Start the program.
2. Import the Regular Expression module using import re.
3. Store product names in a list.
4. Get a search keyword from the user.
5. Perform an exact search using re.fullmatch().
6. Perform a prefix search using re.match().
7. Perform a suffix search using re.search() with $.
8. Perform a partial search using re.search().
9. Use re.I for case-insensitive matching.
10. Display the matching products and their counts.
11. Stop the program.

CODE:

```
import re
products = ["Laptop", "Laptop Bag", "Gaming Laptop","Mouse", "Wireless Mouse", "Laptop Stand"]
keyword = input("Enter keyword: ")
exact = [p for p in products if re.fullmatch(
    re.escape(keyword), p, re.I)]
prefix = [p for p in products if re.match(
    re.escape(keyword), p, re.I)]
suffix = [p for p in products if re.search(
    re.escape(keyword) + r"$", p, re.I)]
partial = [p for p in products if re.search(
    re.escape(keyword), p, re.I)]
for label, result in [
    ("Exact", exact), ("Prefix", prefix),
    ("Suffix", suffix), ("Partial", partial)]:
    print(label, "matches:", result)
    print("Count:", len(result))
```

OUTPUT:

```
Enter keyword: laptop
Exact matches: ['Laptop']
Count: 1
Prefix matches: ['Laptop', 'Laptop Bag', 'Laptop Stand']
Count: 3
Suffix matches: ['Laptop', 'Gaming Laptop']
Count: 2
Partial matches: ['Laptop', 'Laptop Bag', 'Gaming Laptop', 'Laptop Stand']
Count: 4
```

Result Analysis

- Product Input: The program stores product names in a list.
- Keyword Input: The user enters a keyword, such as Laptop.
- Exact Search: re.fullmatch() finds products whose entire name matches the keyword. Result: 1 product.
- Prefix Search: re.match() finds products whose names begin with the keyword. Result: 3 products.
- Suffix Search: re.search() with $ finds products whose names end with the keyword. Result: 2 products.
- Partial Search: re.search() finds products containing the keyword anywhere in the name. Result: 4 products.
- Case-Insensitive Search: re.I allows matching regardless of uppercase or lowercase letters.
- Result: The system successfully displays matching products and their counts for all four search types.

Section 3: Regular Expressions - University Registration System

Algorithm

1. Start the program.
2. Import the Regular Expression module using `import re`.
3. Get the register number, institutional email, course code, semester, and mobile number.
4. Validate the register number using `r"^[A-Z]{2,3}\d{2}[A-Z]?\d{3,4}$"`.
5. Validate the email using `r"^[\w.-]+@saveetha\.com$"`.
6. Validate the course code using `r"^[A-Z]{2,4}\d{3}$"`.
7. Validate the semester using `r"^[1-8]$"`.
8. Validate the mobile number using `r"^[6-9]\d{9}$"`.
9. Use `re.fullmatch()` to validate each field.
10. Display the validity of each field.
11. Display Registration Successful if all fields are valid; otherwise, display Registration Failed.
12. Stop the program.

CODE:

```
import re
reg = input("Register number: ")
email = input("Institutional email: ")
course = input("Course code: ")
semester = input("Semester: ")
mobile = input("Mobile number: ")
patterns = {
    "Register number": r"[A-Z]{2,3}\d{2}[A-Z]?\d{3,4}",
    "Email": r"[\w.-]+@saveetha\.com",
    "Course code": r"[A-Z]{2,4}\d{3}",
    "Semester": r"[1-8]",
    "Mobile": r"[6-9]\d{9}"
}
values = [reg, email, course, semester, mobile]
valid = []
for (field, pattern), value in zip(patterns.items(), values):
    result = bool(re.fullmatch(pattern, value))
    valid.append(result)
    print(field + ":", "Valid" if result else "Invalid")
```

```
print("Registration Successful" if all(valid)
    else "Registration Failed")
```

OUTPUT:

```
Register number: ABC1A1234
Institutional email: jothika@saveetha.com
Course code: DSA0308
Semester: 7
Mobile number: 9876543210
Register number: Invalid
Email: Valid
Course code: Invalid
Semester: Valid
Mobile: Valid
Registration Failed
```

Result Analysis

1. Input Collection: The program accepts the register number, institutional email, course code, semester, and mobile number.
2. Register Number Validation: It checks whether the register number follows the required letter-and-digit format.
3. Email Validation: It verifies that the email ends with @saveetha.com.
4. Course Code Validation: It checks the required uppercase letters followed by three digits.
5. Semester Validation: It accepts semester numbers from 1 to 8.
6. Mobile Validation: It checks whether the mobile number contains ten digits and begins with a digit from 6 to 9.
7. Result Checking: re.fullmatch() verifies each complete input against its pattern.
8. Final Status: If all five fields are valid, the system displays Registration Successful. If any field is invalid, it displays Registration Failed.